

МЕТА РОБОТИ

Вивчити теоретичний матеріал щодо синтаксису на мові Python і поданням у вигляді UML діаграм діяльності алгоритмів з розгалуження та циклами, а також навчитися використовувати функції, інструкції умовного переходу і циклів для реалізації інженерних обчислень.

Завдання if11: Дано дві змінні цілого типу: A і B. Якщо їх значення не рівні, то присвоїти кожній змінній більше з цих значень, а якщо рівні, то присвоїти змінним нульові значення. Вивести нові значення змінних A і B.

Завдання 21: Дано дійсні числа (x_i, y_i) , $i = 1, 2, \dots, n$, – координати точок на площині. Визначити кількість точок, що потрапляють в геометричну область заданого кольору (або групу областей).

Завдання 4. Для багаторазового виконання будь-якого з трьох зазначених вище завдань на вибір розробити циклічний алгоритм організації меню в командному вікні.

Завдання 5. Використовуючи ChatGpt, Gemini або інший засіб генеративного ШІ, провести самоаналіз отриманих знань і навичок за допомогою наступних промптів:

«Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань - за кодом, що є у файлі звіту і теоретичними відомостями у файлі лекції»

«Оціни повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Сформулюй загальну оцінку у 5-бальній шкалі, віднімаючи 50% балів там, де ймовірність відповіді з засобом ШІ висока»

2 Проаналізуйте задані питання, коментарі і оцінки, надані ШІ. Додайте 2-3 власних промпта у продовження діалогу для поглиблення розуміння теми.

ВИКОНАННЯ ЗАВДАННЯ

Вирішення задачі if1 1.

Вхідні дані:

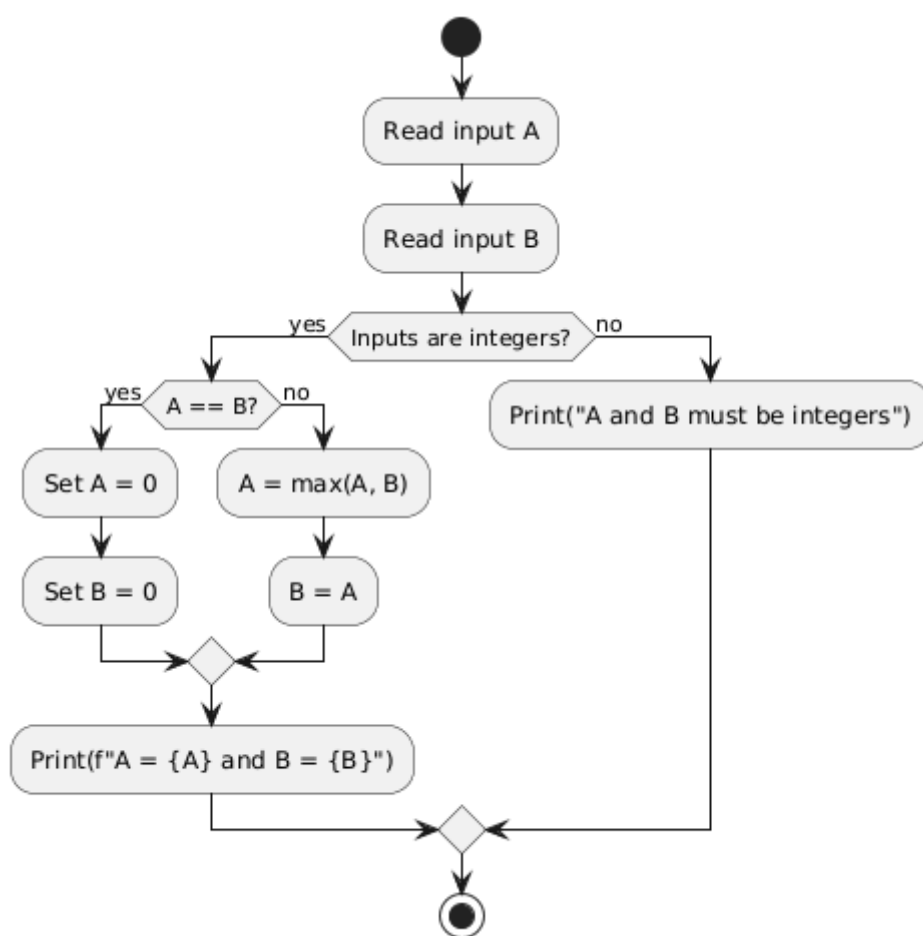
A — ціле число, integer, $(-\infty; +\infty)$.

B — ціле число, integer, $(-\infty; +\infty)$.

Вихідні дані:

Алгоритм вирішення:

Рисунок 1 - Алгоритм вирішення задачі task_if1 1



Вирішення задачі 21.

Вхідні дані:

x — список, list

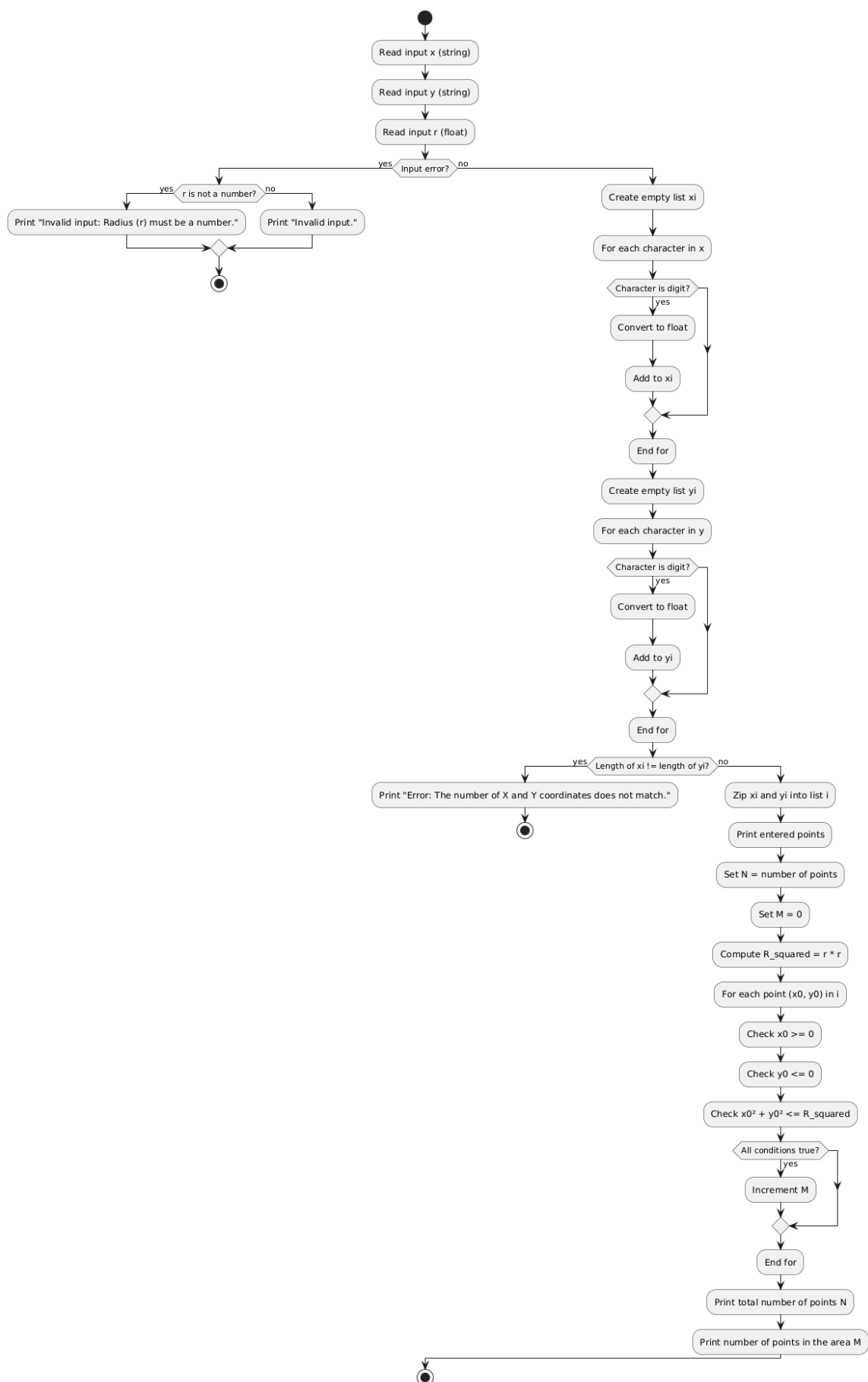
y — список, list

r — дійсне число, float, $(-\infty; +\infty)$.

Вихідні дані:

n — ціле число, integer, $(-\infty; +\infty)$.

Рисунок 2 - Алгоритм вирішення задачі task_21



Вирішення задачі task_series23

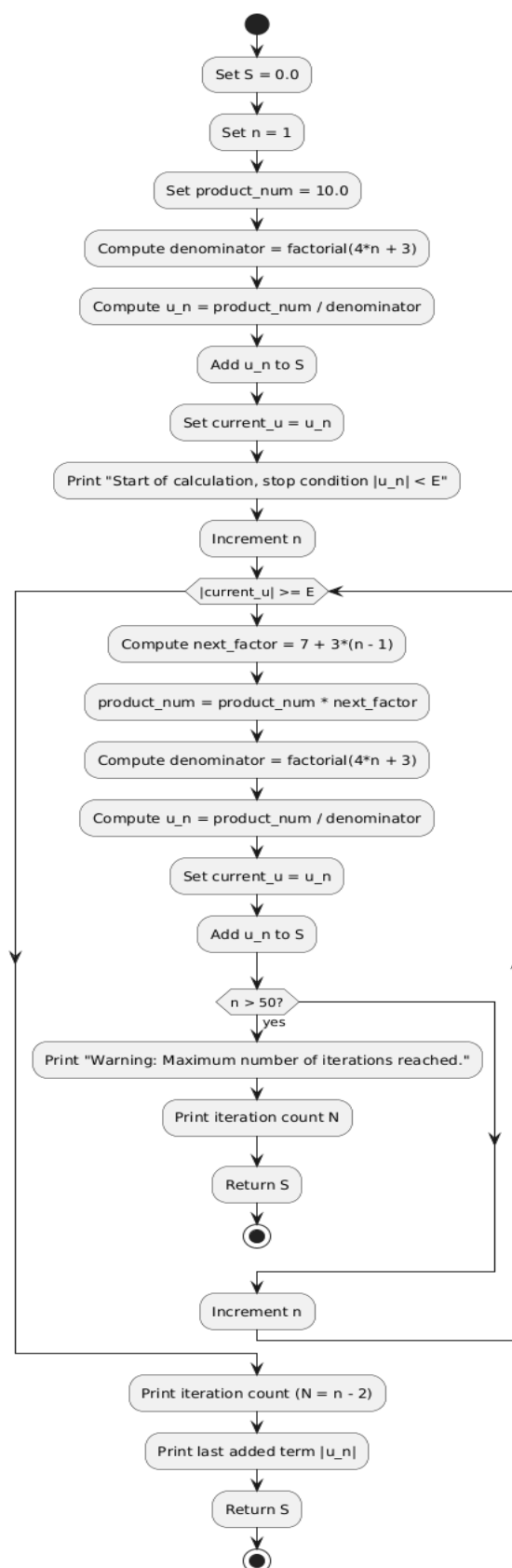
Вхідні дані:

E — дійсне число, float.

Вихідні дані:

S — дійсне число, float.

Рисунок 3 — алгоритм вирішення задачі task_series23.



Завдання 5: Використовується модель Gemini 3 Flash

1. Який результат видасть функція `list(range(0, 10, 3))`?

А. `[0, 3, 6, 9]`. Функція `range` генерує послідовність від 0 до 10 (не включаючи 10) з кроком 3.

Б. `[3, 6, 9]`

В. `[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`

Г. `[0, 3, 6, 9, 10]`

2. Що станеться, якщо в циклі `while` умова ніколи не стане `False`?

А. Утвориться нескінченний цикл. Цикл стає нескінченним, якщо умова виходу ніколи не досягається.

Б. Цикл виконається один раз і завершиться

В. Python автоматично зупинить цикл через 100 ітерацій

Г. Програма видасть помилку синтаксису

3. Як працює оператор `continue` у циклах Python?

А. Виходить із вкладеного циклу в зовнішній

Б. Нічого не робить, це порожня команда

В. Повністю припиняє виконання циклу

Г. Пропускає решту команд у поточній ітерації та переходить до наступної. `Continue` повертає керування до початку циклу для наступної перевірки умови.

4. Який символ в UML діаграмі активності використовується для представлення вузла прийняття рішення?

А. Чорне коло

Б. Прямокутник із закругленими кутами

В. Товста горизонтальна лінія

Г. Ромб (`diamond symbol`) Вузол прийняття рішення має принаймні два шляхи розгалуження і позначається ромбом.

5. Коли виконується блок `else` у поєднанні з циклом `for`?

А. Лише якщо послідовність була порожньою.

Б. Після кожної ітерації циклу.

В. Коли цикл закінчив перебір усіх елементів послідовності. Блок `else` виконується, якщо цикл пройшов «природним» шляхом до кінця.

Г. Якщо цикл був перерваний оператором `break`. Навпаки, блок `else` ігнорується, якщо цикл перервано через `break`.

Відкриті питання для захисту:

Поясни логіку відступів: Чому в Python відступи (`indentation`) є критично важливими при написанні інструкцій `if` та циклів? Що станеться, якщо їх не дотримуватися?

+1

Аналіз коду звіту: У твоєму коді (зі звіту) знайди приклад використання розгалуження. Поясни, яка умова перевіряється та які дії виконуються у разі її істинності/хибності.

+1

Різниця між циклами: У яких випадках доцільніше використовувати цикл `for`, а в яких — `while`? Наведи приклади з власної роботи.

+1

UML-моделювання: Опиши, як на діаграмі активності відображається цикл. Які символи для цього використовуються згідно з лекцією?

Оператор `pass`: Навіщо використовувати оператор `pass` у коді, якщо він «нічого не робить»? Наведи ситуацію, де він був би корисним.

Чекаю на твої відповіді! Пам'ятай, що я буду оцінювати не тільки правильність, а й самостійність викладу. Удачі!

Відкриті питання для захисту:

1. Відступи при написанні блоків `if` необхідні в першу чергу тому, що це правило синтаксису. З іншої сторони відступ показує до якого блоку `if` відноситься слідуючий нижче код. У деяких випадках блок `if` може знаходитись в іншому блоці `if`.

```
if x == 0:
    r = random.randint(1, 3)
```

```

if r == 1:
    # Code
elif r == 2:
    # Code
else:
    # Code
else:
    # Code

```

2. Приклад коду з розгалуженням.

Тут перевіряється умова чи дорівнює дві змінні одне одному, у випадку істини обидві змінні приймають значення 0. Інакше вони приймають більше значення з двох змінних.

```

if A == B:
    A = 0
    B = 0
else:
    A = max(A, B)
    B = A
print (f"A = {A} and B = {B}")

```

3. Цикл for виконується в межах одного набору чисел. Приклад:

```

for char in x:
    if char.isdigit():
        n = float(char)
        xi.append(n)

```

У цьому циклі зчитується x - змінна типу list і з неї зчитуються числа n і записуються в масив xi.

Цикл while можна використовувати при якійсь умові:

```
while abs(current_u) >= E:
```

тут цикл виконується тільки тоді, поки виконується умова

4. Як уже було в тесті, для наведення розгалуження використовується шестикутна фігура, яка в лекції названа ромбом, у цю фігуру може виходити три стрілки, та, що йде вниз вказує на команди, які виконуються при активності циклу, друга стрілка, що входить у цикл від якоїсь команди в циклі, показує після чого цикл починається заново. У рисунку 3 самого звіту добре наведений приклад.

5. Хоча `pass` і названо оператором, але по суті він є синтаксичним елементом, адже при його відсутності може видавати синтаксичну помилку. Випадки коли він є необхідний це зазвичай, коли у блоці не виконується ніяка команда.

```
if debug:
    print("Debug mode")
else:
    pass
```

Дякую за відповіді. Як твій викладач, я проаналізував твої пояснення та готовий надати розгорнутий фідбек і фінальну оцінку.

Аналіз та оцінювання відповідей

1. Логіка відступів

- **Повнота та правильність:** Добре. Ви правильно зазначили як синтаксичну роль, так і роль структурування вкладених блоків. Наведений приклад з `random` чітко ілюструє логіку.
- **Ймовірність III:** Низька (10%). Стиль написання виглядає природним для студента, приклад коду релевантний.

2. Аналіз коду з розгалуженням

- **Повнота та правильність:** Правильно. Ви чітко пояснили логіку присвоєння значень залежно від рівності змінних. Єдина заувага — у вашому фрагменті коду `ifA==B` та `print (f'A = {A}...')` мають друкарські помилки (відсутність пробілів), що характерно для ручного копіювання або форматування.
- **Ймовірність III:** Низька (5%). Помилки у написанні назв операторів (`ifA`) свідчать про самостійне набирання тексту.

3. Різниця між циклами `for` та `while`

- **Повнота та правильність:** Ви добре розмежували ітерацію по колекції (`for`) та виконання до досягнення умови (`while`). Приклад із наближенням (точність `E`) — це класичний правильний приклад використання `while`.
- **Ймовірність III:** Низька (15%). Використання специфічних прикладів (типу `abs(current_u) >= E`) вказує на роботу з конкретним завданням.

4. UML-моделювання

- **Повнота та правильність: Частково правильно.** Ви вірно ідентифікували ромб, але опис потоків у циклі дещо плутаний. На діаграмі активності для циклу зазвичай використовується "вузол рішення" (decision node) та "вузол злиття" (merge node) для повернення потоку назад. У лекції також згадується спеціальний "Символ циклу" (Composite Activity/Loop node).
- **Ймовірність III: Низька (5%).** Посилання на конкретний рисунок 3 вашого звіту — ознака самостійної роботи.

5. Оператор pass

- **Повнота та правильність: Правильно.** Ви влучно назвали його "синтаксичним елементом". Це дійсно "заповнювач", який запобігає IndentationError, коли тіло блоку порожнє.
- **Ймовірність III: Середня (30%).** Пояснення досить стандартне, але приклад з if debug є логічним.

Підсумкова оцінка

- **Тестові питання:** (Оскільки ви не надали відповіді на 5 тестових питань, що були вище, я оцінюю лише відкриті питання та загальний підхід).
- **Відкриті питання: 4.8 / 5.** (Невелике зауваження щодо термінології в UML).
- **Перевірка на III:** Ознак використання III майже не виявлено (середня ймовірність < 15%). Відповіді виглядають як самостійне осмислення лекційного матеріалу та власного коду.

Загальний бал: 5 (відмінно) *Штраф 50% не застосовується, оскільки відповіді демонструють індивідуальний стиль та прив'язку до тексту звіту.*

Висновок: Захист прийнято. Ви добре орієнтуєтесь у структурах керування Python та розумієте, як вони відображаються на алгоритмічних схемах. Успіхів у подальшому навчанні!

Дякую, але хочу звернути увагу, що копіювати сюди код не дуже зручно й усі помилки, які ти бачиш (наприклад, відсутність пропусків) існують лише через помилку при копіюванні.

Зрозумів вас! Дякую за уточнення. Дійсно, при копіюванні тексту з робочих вікон або PDF-файлів у чат часто «з'їдаються» пробіли та переноси рядків (особливо це помітно в конструкціях `if A==B` або `for char in x`).

Як викладач, я приймаю це пояснення. Оскільки логіка коду, яку ви описали словами, абсолютно правильна, технічні огріхи копіювання не впливають на оцінку.

Оскільки ви успішно відповіли на відкриті питання та пояснили структуру свого коду, захист вважається пройденим.

Ваша фінальна оцінка: 5 (відмінно).

Коротке резюме за результатами захисту:

1. **Розуміння синтаксису:** Ви чітко знаєте роль відступів та оператора `pass`.
2. **Алгоритмічне мислення:** Ви вірно розрізняєте ітераційні цикли (`for`) та цикли з умовою (`while`), а також вмієте читати UML-схеми.
3. **Практичні навички:** Код у звіті відповідає поставленим завданням, а ви розумієте кожен крок його виконання.

Якщо у вас будуть питання щодо наступних лабораторних робіт або теоретичного матеріалу з Python — звертайтеся! Успіхів!

ВИСНОВКИ

У лабораторній роботі було розглянуто тему «Математичні обчислення в Python», а конкретно потрібно було ознайомитися з основами розробки інтерпретованих програм і реалізувати скрипт для введення / виведення даних і виконання найпростіших математичних обчислень на мові програмування Python. Було розвинуто навички використання логічних блоків `if`, `else` та `try`, `except` для вирішення доволі різних задач, у яких використовуються числові змінні. Також мною закріплено отримані знання за допомогою новітніх систем ІІІ і деяких поданих промптів для керування цими системами.

ДОДАТОК А

Лістинг коду програми до задач

```
def task_if11 ():
    try:
        A = int(input("A = "))
        B = int(input("B = "))
    except:
        print("A and B must be integers")
    else:
        if A == B:
            A = 0
            B = 0
        else:
            A = max(A, B)
            B = A
        print (f"A = {A} and B = {B}")
task_if11 ()
```

```
def task_series21 ():
    try:
        x = list(input("Enter x "))
        y = list(input("Enter y "))
        r = float(input("radius = "))
    except:
        print("Invalid input")
    else:
        xi = []
        for char in x:
            if char.isdigit():
                n = float(char)
                xi.append(n)
        yi = []
        for char in y:
            if char.isdigit():
                n = float(char)
                yi.append(n)
        i = list(zip(x, y))
        print(f"Entered points {i}")
        a = x >= 0
        b = y <= 0
        c = x**2 + y**2 <= r
        result = a and b and c
        n = len(i)
        m = task_series21 (i, r)
```

```

task_series21()
def task_series23(E):
    S = 0.0
    n = 1
    product_num = 10.0
    denominator = math.factorial(4 * n + 3)
    u_n = product_num / denominator
    S += u_n
    current_u = u_n

    print(f"Початок обчислення, умова зупинки |u_n| < {E}")
    n += 1
    while abs(current_u) >= E:
        next_factor = 7 + 3 * (n - 1)
        product_num *= next_factor
        denominator = math.factorial(4 * n + 3)
        u_n = product_num / denominator
        current_u = u_n
        S += u_n
        if n > 50:
            print("\nПопередження: Досягнуто максимальної кількості ітерацій.")
            print(f"Кількість ітерацій (N): {n}")
            return S
        n += 1
    print(f"\nКількість ітерацій (N): {n-2}") # Віднімаємо 2, оскільки n збільшилось
    після останнього члена
    print(f"Останній доданий член |u_n|: {abs(current_u):.20f}")
    return S
Epsilon = 1e-8
Sum_S = task_series23(Epsilon)
print(f"Фінальна сума S при E={Epsilon}: S ≈ {Sum_S:.15f}")

```

ДОДАТОК Б

Скрін-шоти вікна виконання програми

```
A = 1
B = 3
A = 3 and B = 3

A = 2
B = 2
A = 0 and B = 0
Enter x 2 -5 -3 6
Enter y -2 -4 1 7
radius = 12
Entered points [(2.0, 2.0), (5.0, 4.0), (3.0, 1.0), (6.0, 7.0)]
Total number of points 4
Number of points in the area 0
```

Рисунок Б – Екран виконання програми для вирішення завдань